Análisis y Diseño de Algoritmos I

Valeria Bengolea
Departamento de Computación
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Universidad Nacional de Río Cuarto

Técnicas de Diseño: Programación Dinámica

Notas basadas en el material preparado por el Dr. Nazareno Aguirre

Estrategias de Diseño de Algoritmos

Las estrategias de diseño de algoritmos nos brindan herramientas para atacar el problema de construir soluciones algorítmicas para problemas. Hemos visto ya algunas de estas estrategias, Fuerza Bruta, Divide & Conquer y Decrease & Conquer.

Veremos ahora una nueva estrategia denominada Programación Dinámica, que puede entenderse como una optimización a soluciones recursivas,

- Basada en la inversión del cálculo de las soluciones a subproblemas inducida por las invocaciones recursivas de una solución.

Números de Fibonacci

Consideremos el ya conocido problema de computar el n-ésimo número de Fibonacci. Hacer esto recursivamente es simple:

```
fib :: Int -> Int
fib 0 = 1
fib 1 = 1
fib n = (fib (n-1)) + (fib (n-2))
```

$$T(n) = T(n-1) + T(n-2) + 1$$

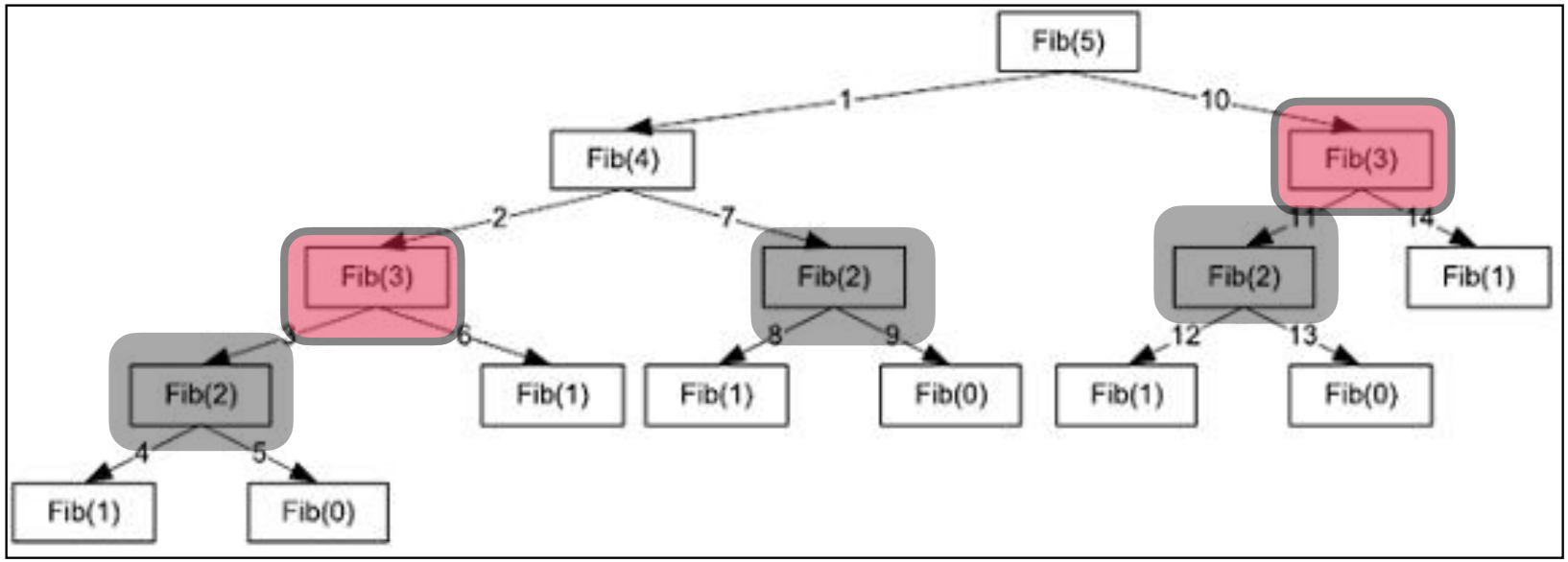
$$O(2^n)$$

Problema:

Resolver fib n demanda recomputar muchas veces un cantidad importante de subproblemas comunes al cálculo de fib (n-1) y fib (n-2).

Ejemplo: Números de Fibonacci

Veamos, por ejemplo, que con 5 como parámetro para fib, ya tenemos una cantidad significativa de problemas que deben recomputarse:



Programación Dinámica

Dado un problema P, para el cual:

- ya se cuenta con una solución Divide & Conquer (top-down, en el sentido que para resolver el problema lo divide en subproblemas más pequeños, y combina sus soluciones para resolver el problema original), PERO
- para cada problema particular,
  - los subproblemas a resolver NO son independientes, sino que comparten a su vez subproblemas (es decir, se "solapan"), y por lo tanto requieren recomputar varias veces las soluciones a algunos subproblemas comunes.
  - el número total de subproblemas a resolver está acotado por una cantidad polinomial, respecto al tamaño del problema que los origina.

Programación Dinámica

Resolver el problema mediante Programación Dinámica consiste en:

> Invertir el orden de cómputo de soluciones a subproblemas de P (es decir, computarlas bottom-up), y almacenar los resultados de manera tal de no tener que recomputarlos.

Programación Dinámica cambia eficiencia por espacio de almacenamiento, pero en muchos casos este intercambio es extremadamente beneficioso.

Números de Fibonacci mediante Programación Dinámica

Si pensamos en resolver Fibonacci mediante Programación Dinámica, debemos invertir el orden de cómputo de soluciones a subproblemas (desde los casos base hacia arriba), y almacenar estas soluciones para construir las soluciones a problemas más "grandes". Una forma directa de hacer esto es la siguiente:

ALGORITHM *Fib*(n)

//Computes the nth Fibonacci number iteratively by using its definition
//Input: A nonnegative integer n
//Output: The nth Fibonacci number
$F[0] \leftarrow 0;\ F[1] \leftarrow 1$
for $i \leftarrow 2$ **to** n **do**
    $F[i] \leftarrow F[i-1] + F[i-2]$
return $F[n]$

$$T(n) = \sum_{i=2}^{n} 1$$

$$O(n)$$

Y...¿espacio ocupado?

Números de Fibonacci mediante Programación Dinámica

Con respecto al espacio ocupado, la versión mediante Programación Dinámica es O(n). Podemos mejorar aún más este algoritmo, pues sólo necesitamos los dos últimos valores almacenados en la lista, y no la lista completa:

```
public static int fibonacci(int n) {
     if(n == 0) {
          return 0;
       }
       else {
          int fibAcum0 = 0;
          int fibAcum1 = 1;
          int aux;

          for (int i = 1; i < n; i++)
              {
                  aux = fibAcum0 + fibAcum1;
                  fibAcum0 = fibAcum1;
                  fibAcum1 = aux;
              }

          return fibAcum1;
       }
}
```

Ejemplo: Cálculo de la Combinatoria

Computar $\binom{n}{m} con\ n \geq 0\ y\ m \geq 0$

Número de posibles conjuntos de m elementos distintos que se pueden extraer de un conjunto de n elementos (m tomados de n)

Ejemplo: Combinatoria

$$\binom{n}{0} = 1$$

$$\binom{n}{n} = 1$$

$$\binom{n}{m} = 0 \ si \ m > n$$

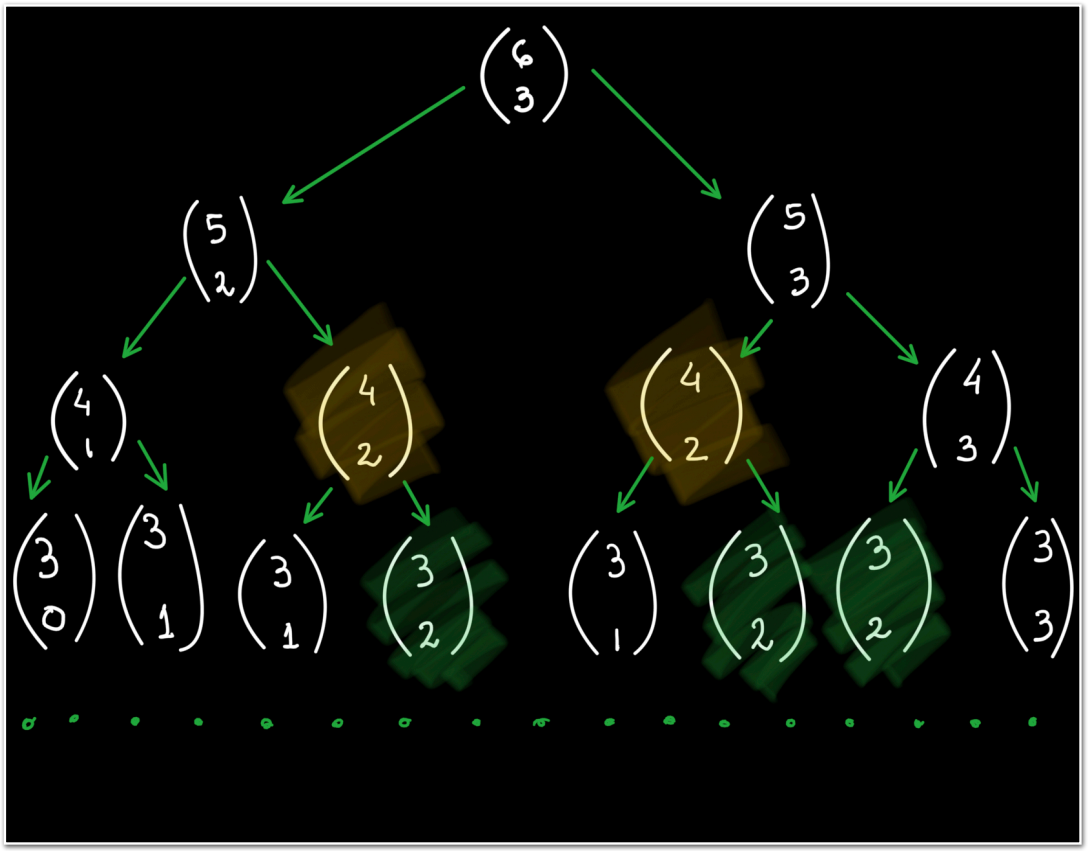
$$otherwise: \ \binom{n}{m} = \binom{n-1}{m-1} + \binom{n-1}{m}$$

Ejemplo: Combinatoria

```
comb:: Int ->Int -> Int
comb n m | n<0  || m <0 = error "error"
         | m >n  = 0
         | m == 0 || n == m = 1
         | otherwise = (comb (n-1)(m-1)) + (comb (n-1) m)
```

```
public static int comb(int n, int m) {
   if (n<0  || m <0)
     throw new IllegalArgumentException("error");
   if (m > n) return 0;
   if (m == 0 || n == m) return 1;
   return comb((n-1),(m-1)) + comb ((n-1),m);
 }
```

Ejemplo: Combinatoria



Ejemplo: Combinatoria

Enfoque bottom-up

Inicio con los Casos Bases

```
comb:: Int ->Int -> Int
comb n m | n<0  || m <0 = error "error"
         | m >n  = 0
         | m == 0 || n == m = 1
         | otherwise = (comb (n-1)(m-1)) + (comb (n-1) m)
```

	0	1	2	3	4	5				m
0	1									
1	1	1								
2	1		1							
3	1			1						
4	1				1					
5	1					1				
6	1						1			
	1							1		
	1								1	
	1									1
	1									
n										

$m < n$

Ejemplo: queremos calcular:

comb n m

necesitamos

comb n-1 m-1 y comb n-1 m

Ejemplo: Combinatoria

¿Cómo poblamos la matriz?

```
comb:: Int ->Int -> Int
comb n m | n<0  || m <0 = error "error"
         | m >n  = 0
         | m == 0 || n == m = 1
         | otherwise = (comb (n-1)(m-1)) + (comb (n-1) m)
```

	0	1	2	3	4	5				m
0	1									
1	1	1								
2	1		1							
3	1			1						
4	1				1					
5	1					1				
6	1						1			
	1							1		
	1								1	
	1									1
n	1									

$m < n$

Ejemplo: queremos calcular:

comb 6 3

necesitamos

comb 5 2 y comb 5 3

Ejemplo: Combinatoria

```java
public static int comb(int n, int m) {
    if (n<0  || m <0)
      throw new IllegalArgumentException("error");
    if (m > n) return 0;
    if (m == 0 || n == m) return 1;
    return comb((n-1),(m-1)) + comb ((n-1),m);
  }
```

$$T(n,m) = \sum_{i=2}^{n} \left(\sum_{j=1}^{\min(m,i-1)} c_2 \right)$$

¿Cambia si la operación de interés es la operación de escritura en la matriz?

$$T(n,m) = \sum_{i=0}^{n} c_1 + \sum_{i=2}^{n} \left(\sum_{j=1}^{\min(m,i-1)} c_2 \right)$$

Enfoque bottom-up

```java
public static int comb(int n, int m) {
    if (n<0  || m <0)
      throw new IllegalArgumentException("error");
    if (m > n) return 0;

    int [][] comb = new int[n+1][m+1];

    for(int i = 0; i <=n; i++) {
      comb[i][0] = 1;
      if (i<=m) {
        comb[i][i] = 1;
      }
    }

    for(int i = 2; i <=n; i++) {
      for(int j = 1; j <=m && j < i ; j++) {
        comb[i][j] =comb[i-1][j-1] + comb[i-1][j];
      }
    }
    return comb[n][m];
  }
```

Operaciones de interés +

Ejemplo: Combinatoria

$$
\begin{aligned}
T(n,m) &= \sum_{i=2}^{n}\left(\sum_{j=1}^{\min(m,i-1)} c_2\right) \\
&= \sum_{i=2}^{m}\left(\sum_{j=1}^{i-1} c_2\right) + \sum_{i=m+1}^{n}\left(\sum_{j=1}^{m} c_2\right) \\
&= c_2 \sum_{i=2}^{m}(i-1) + c_2 \sum_{i=m+1}^{n} m \\
&= c_2\left[\frac{(m-1)m}{2}\right] + c_2\left[m\cdot(n-m)\right] \\
&= c_2\left(\frac{m^2-m}{2} + n\cdot m - m^2\right) \\
&= c_2\left(n\cdot m - \frac{m^2}{2} - \frac{m}{2}\right) \implies \Theta(n\cdot m)
\end{aligned}
$$

Enfoque bottom-up

```java
public static int comb(int n, int m) {
    if (n<0  || m <0)
      throw new IllegalArgumentException("error");
    if (m > n) return 0;

    int [][] comb = new int[n+1][m+1];

    for(int i = 0; i <=n; i++) {
      comb[i][0] = 1;
      if (i<=m) {
        comb[i][i] = 1;
      }
    }

    for(int i = 2; i <=n; i++) {
      for(int j = 1; j <=m && j < i ; j++) {
        comb[i][j] =comb[i-1][j-1] + comb[i-1][j];
      }
    }
    return comb[n][m];
  }
```

Operaciones de interés +

Ejemplo: Cálculo de "Distancia de Edición" entre dos Cadenas

Consideremos el siguiente problema:

Dadas dos cadenas s1 y s2, calcular el número mínimo de operaciones que permiten transformar s1 en s2, donde las operaciones admisibles son:

- reemplazar un caracter particular en una cadena por otro caracter
- insertar un nuevo caracter en una posición dada de alguna de las cadenas
- eliminar un caracter de una posición dada en alguna de las cadenas

La distancia de edición (o distancia de Levenshtein) tiene numerosas aplicaciones, entre ellas en diccionarios para procesadores de textos (e.g., para sugerir palabras alternativas cuando se encuentra una que no pertenece al diccionario), procesos de ayuda en buscadores, etc.

Ejemplo: Cálculo de "Distancia de Edición" entre dos Cadenas

Primero, resolvamos el problema de calcular la distancia de edición recursivamente:

```
distance :: String -> String -> Int
distance xs [] = length xs
distance [] xs = length xs
distance (x:xs) (y:ys)
  | x == y    = min (distance xs ys) (min ((distance (x:xs) ys)+1)
((distance xs (y:ys))+1))
  | otherwise = min ((distance xs ys)+1) (min ((distance (x:xs) ys)+1)
((distance xs (y:ys))+1))
```

Este algoritmo recursivo es extremadamente ineficiente.

Ejemplo: Cálculo de “Distancia de Edición” entre dos Cadenas

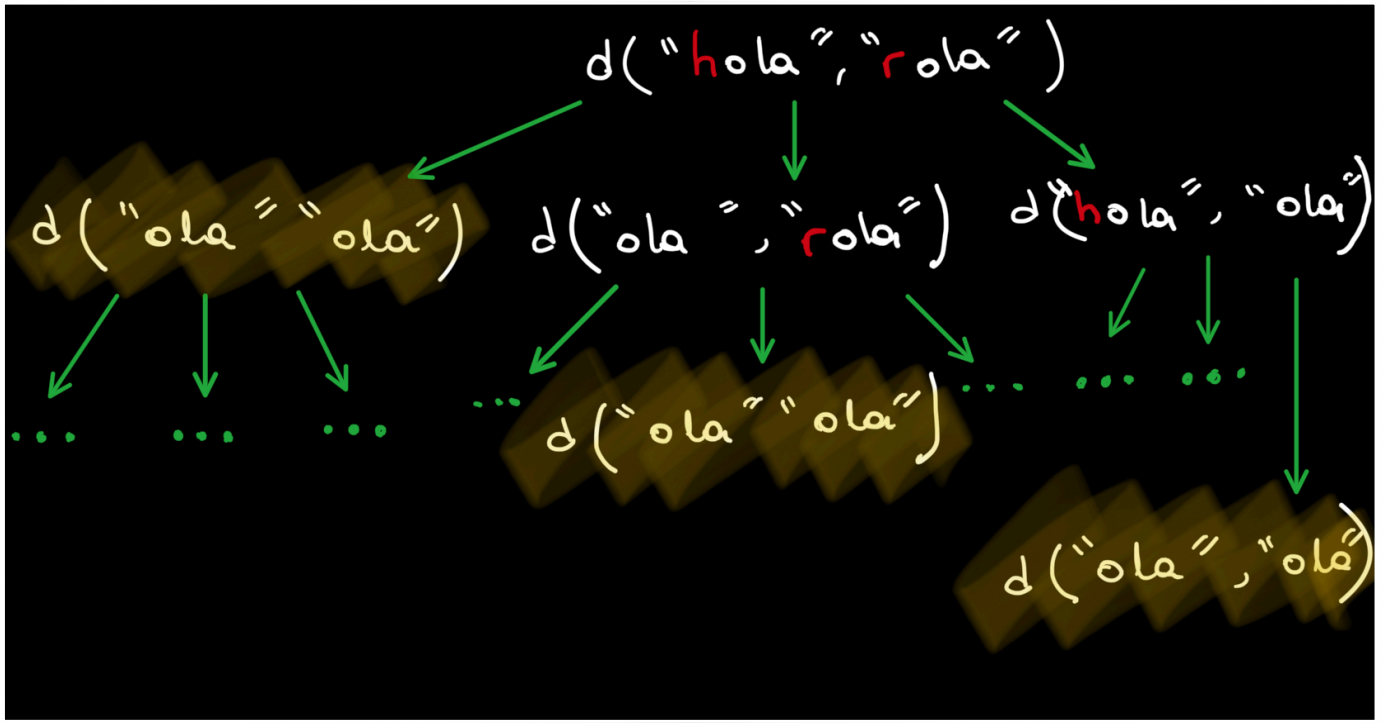
```
distance(gato, pato)
```

distance(“gato”, “pato”)			
Posibilidades			
Sustituir ‘g '→'p'	(+1)	→	distance (“ato”, “ato")
Borrar 'g'	(+1)	→	distance (“ato” “pato")
Insertar 'p'	(+1)	→	distance (“gato”, “ato")



distance(“ato”, “ato”)			
Posibilidades			
	(+0)	→	distance (“to”, “to")
Borrar ‘a'	(+1)	→	distance (“to” “ato")
Insertar ‘a'	(+1)	→	distance (“ato”, “to")

Cálculo de “Distancia de Edición” mediante Programación Dinámica



Cálculo de “Distancia de Edición” mediante Programación Dinámica

Intentemos resolver el problema anterior mediante Programación Dinámica.
Necesitamos invertir el orden de cómputo de soluciones a subproblemas, y hacerlo “bottom-up”.

Cuáles son los subproblemas?

- pares de subcadenas de las cadenas s1 y s2 (¿todas?)

¿En qué almacenamos las soluciones?

- en una matriz, indexada por pares de subcadenas de s1 y s2

¿Cómo “recorremos” la matriz para “poblarla” con valores de distancias?

Cálculo de "Distancia de Edición" mediante Programación Dinámica

Almacenaremos las soluciones a subproblemas en una matriz. Notemos que sólo necesitamos computar distancias de sufijos de las cadenas originales del problema.

	0	1	2	3	4	5	6	7	8	9	10
0	0	1	2	3	4	5	6	7	8	9	10
1	1										
2	2										
3	3										
4	4										
5	5										
6	6										
7	7										
8	8										
9	9										
10	10										

Longitud de la cadena s2

Pensemos en los casos bases:

```
distance xs [] = length xs
distance [] xs = length xs
```

Ejemplo:
D[0,3] = distancia de edición entre la cadena vacía y los últimos 3 caracteres de la segunda cadena = 3

D[i,j]: distancia de edición entre los últimos i caracteres de s1 y los últimos j caracteres de s2.

Longitud de la cadena s1

```haskell
distance :: String -> String -> Int
distance xs [] = length xs
distance [] xs = length xs
distance (x:xs) (y:ys)
  | x == y    = min (distance xs ys) (min ((distance (x:xs) ys)+1)
((distance xs (y:ys))+1))
  | otherwise = min ((distance xs ys)+1) (min ((distance (x:xs) ys)+1)
((distance xs (y:ys))+1))
```

Alm
nec

	0	1	2	3	4	5	6	7	8	9	10
0	0	1	2	3	4	5	6	7	8	9	10
1	1										
2	2										
3	3										
4	4										
5	5										
6	6										
7	7										
8	8										
9	9										
10	10										

D[i,j]: distancia de edición entre los últimos i caracteres de s1 y los últimos j caracteres de s2.

¿Cómo llenamos el resto de la matriz?

Para calcular D[i, j] que necesito tener previamente calculado?

D[i, j-1], D[i-1, j] y D[i-1, j-1]

¿Dónde buscamos la solución al problema?

Cálculo de "Distancia de Edición" mediante Programación Dinámica

El siguiente programa resuelve el problema de calcular la distancia de edición entre dos cadenas:

```
public static int computeLevenshteinDistancePD(String str1, String str2) {
    int[][] distance = new int[str1.length()+1][str2.length()+1];

    for(int i=0; i<=str1.length(); i++){
      distance[i][0] = i;
    }
    for(int j=0; j<str2.length()+1; j++)
      distance[0][j]=j;

    for(int i=1; i<=str1.length(); i++)
      for(int j=1;j<=str2.length(); j++) {
        distance[i][j]= minimum(distance[i-1][j]+1,
                      distance[i][j-1]+1,
                      distance[i-1][j-1]+((str1.charAt(i-1)==str2.charAt(j-1))?0:1));
      }

    return distance[str1.length()][str2.length()];
  }
```

Programación Dinámica en Haskell

Números de Fibonacci

```haskell
fib :: Int -> Int
fib n = head (fibList n)

fibList :: Int -> [Int]
fibList n | n==0 = [1]
          | n==1 = [1,1]
          | n>1  = ((head l)+(head (tail l))):l
          where l = fibList (n-1)
```

```haskell
fib :: Int -> Int
fib n = fst (fibTuple n)

fibTuple :: Int -> (Int,Int)
fibTuple n  | n == 0 = (1,0)
            | n == 1 = (1,1)
            | n > 1 = ((fst t)+(snd t),fst t)
            where t = fibTuple (n-1)
```

Resumen de la Clase

- La Programación Dinámica es una técnica de diseño de algoritmos fuertemente ligada a recursión/D&C, que intenta mejorar la eficiencia de tales soluciones invirtiendo el orden de cómputo de soluciones a subproblemas.
- El cómputo de soluciones se realiza de manera “bottom-up” (desde los casos base hacia los casos más complejos), en lugar del típico mecanismo “top-down” correspondiente a la recursión.
- El uso de Programación Dinámica es un ejemplo importante de intercambio de eficiencia en tiempo por eficiencia en espacio.

Resumen de la Clase

- En casos en que el almacenamiento de soluciones a subproblemas es realizado sobre una estructura del estilo de arreglos, listas o matrices, debe asegurarse que la carga de la estructura con valores se haga en un orden adecuado: al computar un valor dado ya deben haberse computado los valores correspondientes a los subproblemas del problema corriente.
- La Programación Dinámica es una técnica particularmente útil en problemas de optimización.